WEB-250 | asgn08 - Active Record Directory

Course: WEB-250 Database-Driven Website
Points: 100 (Active Record assignment only)
Due: As posted in Moodle
Starting point: Your completed **asgn07** WNC Birds project
Learning source: Kevin Skoglund, *PHP Object-Oriented Programming with Databases*, **Chapter 3, videos 3.1-3.6**

> **Naming note:** The LinkedIn Learning materials are **Chapter 3**. This assignment in **WEB-250** is **asgn08**. Keep your project and Git commit names as `asgn08`; do not rename them to chapter03.
>
> **Two separate grades: asgn08 — Active Record Directory (100 points)** and **Debugging Setup — VS Code + Xdebug (100 points)**. The debugging assignment has its own handout, starter ZIP, GitHub/webhost links, and Moodle activity. No debugging points are included in this asgn08 rubric.

1. 📖 Introduction - What is Active Record?

In asgn07, you connected PHP to MySQL with PDO, ran queries, and displayed information. You also created three related tables in the `wnc_birds` database:

- `birds` - species information (the table used for our directory).
- `bird_images` - information about image files.
- `bird_image_links` - which images belong to which birds (a many-to-many junction table).

In asgn08, we will **keep this database** and make PHP code easier to organize. Right now a page often needs to know *how to write the SQL query*, *how to fetch database rows*, and *how to display HTML*. It is difficult to reuse that code in several pages. The **Active Record** idea is to represent a database row with a PHP object and let its class know how to retrieve those rows.

For example:

```php
$birds = Bird::find_all();        // array of Bird objects
$bird = Bird::find_by_id(3);     // one Bird object, or null

if ($bird !== null) {
    print $bird->common_name;     // property of that object
}
```

The code above is a preview, **not** a finished implementation. You will write the methods that make it work. The example ID `3` is only illustrative; IDs in your database may differ.

A quick map of what happens

```
public/birds.php                Bird class                  MySQL
     |                              |                         |
```

```
|-- Bird::find_all() ---------->|                          |
|                               |-- SELECT ... FROM birds ->|
|                               |<-- associative rows ------|
|                               |-- row -> Bird object      |
|<-- array of Bird objects -----|                          |
|-- foreach + escaped HTML      |                          |
```

A **static method**, such as `Bird::find_all()`, is called on the class before you have a specific bird. The properties of a **Bird object**, such as `$bird->common_name`, hold values for one row. When you turn an associative database row into an object, that process is called **hydration**.

We will implement **only the reading portion of Active Record** in this assignment: listing all records and finding a single record. Creating, editing, deleting, authentication, and `.env` configuration are **not** part of asgn08. We are also postponing image galleries; do not remove or change the `bird_images` or `bird_image_links` tables.

Learning goals

By the end, you should be able to:

1. Explain the difference between static model methods and properties on a specific object.
2. Reuse your asgn07 PDO connection inside a model class.
3. Retrieve database rows and turn them into PHP objects.
4. Implement `find_all()` and `find_by_id()` safely with PDO.
5. Refactor duplicate row-to-object code into one helper method.
6. Use a Git development branch, merge to `main`, push to GitHub, and deploy the result to your webhost.

---

2. 🧪 Guided Sandbox - Working code first (**not graded**)

The sandbox is a working example you can experiment with and **keep as a reference** for later assignments. It is deliberately smaller than the independent assignment: the sandbox class only knows `id` and `common_name`, whereas your graded `Bird` class will include the complete bird record.

2.1 Prepare your project

1. Make sure asgn07 works before making changes. Back up your database before making changes to it; **do not** re-import a script that drops existing data.
2. Work in your existing Git repository and **create `dev-active-record` before copying or editing anything**. The Git commands are in Section 5.5.
3. Copy your asgn07 project into a new sibling folder named **`asgn08`**. Leave asgn07 intact.
4. Download **`asgn08_student_starter.zip`**, then extract its *contents* into your new `asgn08` folder. It supplies the complete sandbox and independent-work TODO files. **The separate debugging setup assignment uses its own ZIP in `web250/debugging_setup`.**
5. Confirm you still have your working `private/classes/Database.php` and `private/initialize.php` from asgn07. They are **not** included in the starter ZIP, so they will not overwrite your settings.

Approximate target folder structure:

```
web250/
  asgn07/                          <-- leave working
  asgn08/
    private/
      classes/
        Database.php               <-- from asgn07
        SandboxBird.php            <-- guided sandbox (complete)
        Bird.php                   <-- independent TODOs
      initialize.php               <-- from asgn07
      ... existing configuration and helpers ...
    public/
      index.php                    <-- from asgn07; update it
      sandbox_active_record.php    <-- guided sandbox (complete)
      birds.php                    <-- independent TODOs
      bird.php                     <-- independent TODOs
      images/birds/                <-- from asgn07, if present
    README.md                      <-- update or create
    .gitignore                     <-- update (never add secrets)
```

For a local MAMP installation using port 8888, the sandbox might open at:

`http://localhost:8888/web250/asgn08/public/sandbox_active_record.php`

For XAMPP using its default Apache port, it might be:

`http://localhost/web250/asgn08/public/sandbox_active_record.php`

Your URL may be different depending on your MAMP/XAMPP document root and port. The examples assume your asgn07 project already connects successfully to `wnc_birds`.

2.2 Read `private/classes/SandboxBird.php`

This file is **complete**. Run it before modifying anything. The full code follows so that you can use this document later even without the starter ZIP.

```php
<?php

declare(strict_types=1);

/**
 * UNGRADED GUIDED SANDBOX.
 * A tiny read-only Active Record example using the asgn07 birds table.
 * This class is intentionally smaller than the Bird class in the
assignment.
 */
final class SandboxBird
{
    private static ?PDO $database = null;
```

```
    public int $id = 0;
    public string $common_name = '';

    // Static: one database connection reference shared by this class per
request.
    public static function set_database(PDO $pdo): void
    {
        self::$database = $pdo;
    }

    /** @return SandboxBird[] */
    public static function find_all(): array
    {
        if (self::$database === null) {
            throw new RuntimeException('Call set_database() before
find_all().');
        }

        // No user input is used in this fixed SQL statement.
        $sql = 'SELECT id, common_name FROM birds ORDER BY common_name';
        $statement = self::$database->query($sql);
        $rows = $statement->fetchAll(PDO::FETCH_ASSOC);

        $birds = [];
        foreach ($rows as $row) {
            // Hydration: turn one associative row into one PHP object.
            $bird = new self();
            $bird->id = (int) $row['id'];
            $bird->common_name = (string) $row['common_name'];
            $birds[] = $bird;
        }

        return $birds;
    }
}
```

Read through these statements in particular:

Statement	What it does	Why it matters
`private static ?PDO $database = null;`	Stores a PDO connection reference on the class. It starts with no connection.	Each object does not need its own connection. This shared value lasts within the current PHP request, not forever.
`set_database(PDO $pdo)`	Receives the connection created by the existing `Database` class.	The model does not need the username, password, or connection settings.
`self::$database`	Gets the class's static connection from a static	`self::` is class-level access; `$this->` is for an existing object.

Statement	What it does	Why it matters
	method.	
`query($sql)`	Executes fixed SQL with no user input.	We will use `prepare()` and a bound parameter when an ID comes from a URL.
`fetchAll(PDO::FETCH_ASSOC)`	Returns rows as associative arrays, keyed by column name.	It does **not** automatically return your `SandboxBird` objects.
`new self()` and property assignments	Create one object for each row and copy the values into it.	This is hydration. The integer cast keeps `id` an integer.
`return $birds;`	Gives the page an array of objects.	HTML code can now work with objects rather than raw PDO results.

2.3 Read `public/sandbox_active_record.php`

This page asks the model for its birds and displays them. Notice it does not contain any SQL.

```
<?php

declare(strict_types=1);

require_once dirname(__DIR__) . '/private/initialize.php';
require_once dirname(__DIR__) . '/private/classes/SandboxBird.php';

try {
    // Use the existing asgn07 Database connection class and credentials.
    $pdo = (isset($pdo) && $pdo instanceof PDO) ? $pdo :
Database::connect();
    SandboxBird::set_database($pdo);
    $birds = SandboxBird::find_all();
} catch (PDOException $e) {
    error_log($e->getMessage());
    http_response_code(500);
    exit('Could not load the bird sandbox. Check the database
connection.');
}

// Escape data before inserting it into HTML.
function sandbox_h(string $value): string
{
    return htmlspecialchars($value, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
}
?>
<!doctype html>
<html lang="en">
```

```
<head>
    <meta charset="utf-8">
    <title>Active Record Sandbox</title>
</head>
<body>
    <h1>Active Record Sandbox</h1>
    <p>SandboxBird objects returned: <?= count($birds) ?></p>

    <?php if (count($birds) === 0): ?>
        <p>No bird records were found.</p>
    <?php else: ?>
        <ul>
            <?php foreach ($birds as $bird): ?>
                <li>
                    <?= sandbox_h($bird->common_name) ?>
                    (ID <?= $bird->id ?>)
                </li>
            <?php endforeach; ?>
        </ul>
    <?php endif; ?>
</body>
</html>
```

The `require_once` statements load the previous assignment's setup and our sandbox class. `Database::connect()` supplies a PDO connection, `SandboxBird::set_database($pdo)` shares it with the model, and `SandboxBird::find_all()` returns objects.

On the output side, `htmlspecialchars()` converts special characters in database text into safe HTML text. The code catches PDO connection/query errors, writes technical details to the server's error log, and shows a neutral message to the visitor. Do not print passwords or raw connection errors into a public page.

First test: Open the sandbox in a browser. The original asgn07 seed contains **8** bird records. If you added birds, your count may be higher. On the original data, the alphabetically sorted names are:

```
American Robin
Blue Jay
Carolina Wren
Dark-eyed Junco
Eastern Towhee
Northern Cardinal
Red-bellied Woodpecker
Rose-breasted Grosbeak
```

Your browser page should say `SandboxBird objects returned: 8` and show the names with their record IDs. The IDs themselves may differ if your data has changed. If you get a database error, return to your asgn07 connection configuration and check the MAMP/XAMPP MySQL port (often **8889** with MAMP, though your setup may differ).

---

3. 🔬 Experiment - Predict, change, run, explain (**not graded**)

For each experiment, **write down your prediction before running the code**. Keep your own notes in `sandbox_notes.md` or similar. These notes and the sandbox are **not submitted for a grade**, but keep the code in your repository as a reusable personal example. Restore a working version after each experiment.

Experiment A - Change the sort direction

In `SandboxBird::find_all()`, find `ORDER BY common_name`. Change it to `ORDER BY common_name DESC`.

- **Predict:** Which bird will appear first? What happens to the number of objects?
- **Run:** Refresh the sandbox browser page.
- **Explain:** Did changing the SQL require you to change the HTML loop? Why or why not?
- **Restore:** Change the SQL back to ascending order.

Experiment B - Limit the results

Temporarily add `LIMIT 3` to the end of the query.

- **Predict:** How many `SandboxBird` objects should be returned? Is that the number of rows in the database?
- **Run:** Refresh and compare the displayed count to your prediction.
- **Explain:** Why is `count($birds)` not a reliable way to count the *entire* database once the query has a `LIMIT` clause?
- **Restore:** Remove `LIMIT 3`.

Experiment C - Hydrate an additional property

Extend the sandbox class so each object knows its bird's `scientific_name` as well as its common name.

- Add the property to `SandboxBird`.
- Include its column in `SELECT`.
- Populate the property during the `foreach` hydration loop.
- Display both names in the sandbox HTML.
- **Predict first:** What error or behavior would you expect if you try to access a typed property before giving it a value?
- **Run:** Confirm that each object now has both names.

Keep the successful additional property if it helps you in later assignments.

Experiment D - What if we skip `set_database()`?

Temporarily comment out the `SandboxBird::set_database($pdo);` line in the sandbox page.

- **Predict:** Can the static `find_all()` method run a query with no connection?
- **Run:** Observe the error. **Do not leave the sandbox broken.**

- **Explain:** Why is sharing a PDO connection with the class better than opening a new connection for every row?
- **Restore:** Uncomment the line and confirm the page works again.

---

4. ✅ Checkpoints - Verify what you know

Use this table after the sandbox and again after the graded work. You should be able to answer the questions **without copying definitions from the transcript**.

Checkpoint	Expected behavior / question
A working sandbox	8 names on the original seed data, alphabetized, no warnings or fatal errors.
Result limit	Adding `LIMIT 3` returns **3 objects**, not all rows in `birds`.
Object versus row	Explain why `$bird->common_name` works after hydration but `$row->common_name` does not work on an associative array.
Static versus instance	Explain the difference between `Bird::find_all()` and `$bird->common_name`.
Connection	Which line gives the model its PDO connection? What happens when it is skipped?
Security	Why is a prepared statement more appropriate for a URL-supplied `id`? Why must database text be escaped before displaying it?
Missing record	What should `Bird::find_by_id(2147483647)` return if that ID does not exist?
Refactoring	If two finder methods map the same seven database columns to properties, where should that mapping logic live?

A reminder: after completing the independent assignment, `Bird::find_all()` should return an **array of `Bird` objects**; `Bird::find_by_id()` should return **one `Bird` object or `null`**. Returning a raw `PDOStatement`, a raw associative array, or `false` instead is not the contract for this course assignment.

---

5. 💻 Assignment - Build the Active Record Directory (**graded**)

You have now seen one complete example. The graded work asks you to transfer that idea to the **full** `birds` record and design a details page. Your sandbox can be consulted, but avoid simply copying it without considering how the responsibilities change.

5.1 🔩 Build `private/classes/Bird.php`

The starter file has declarations and TODOs but deliberately **does not implement** the methods. You will create a **read-only model** for all the columns in your asgn07 `birds` table:

PHP property	Database column	Type in your PHP object
`$id`	`id`	integer

PHP property	Database column	Type in your PHP object
`$common_name`	`common_name`	string
`$scientific_name`	`scientific_name`	string
`$family_name`	`family_name`	string
`$primary_habitat`	`primary_habitat`	string
`$seasonal_status`	`seasonal_status`	string
`$description`	`description`	string

Implement these **five** methods in `Bird`:

1. **`set_database(PDO $pdo): void`** - Make the model use the PDO connection already provided by asgn07. Keep the stored connection private and static. Your model should not contain a password or open its own connection for each bird.
2. **`find_all(): array`** - Select all the bird columns, sort by common name, and return an **array of `Bird` objects**. When the table is empty, return `[]`.
3. **`find_by_id(int $id): ?Bird`** - Retrieve a single bird using a **prepared statement** and a bound integer parameter. Return one `Bird` object when found, or `null` when not found; do not use SQL string concatenation with the ID.
4. **`from_row(array $row): Bird`** - A **private helper** that takes one associative row and produces one fully populated Bird object. Both finder methods must use this helper. This is your main refactoring step: there should not be two copies of the same property-assignment block.
5. **`count_all(): int`** - Return `SELECT COUNT(*) FROM birds` as an integer. Do not hydrate a whole collection just to count rows.

Implementation hints, not a solution: `PDO::FETCH_ASSOC` gives you the column names, `new self()` creates an instance in a static class method, `(int)` casts `id`, and `bindValue()` can bind an ID with `PDO::PARAM_INT`. Check the difference between `fetch()` (one row) and `fetchAll()` (many rows). Keep database errors visible in local development, but do not expose raw errors on the public webhost.

Bootstrap: Make sure your shared asgn07 `private/initialize.php` loads `Bird.php` (or your existing autoloader loads it) and calls `Bird::set_database($pdo)` **after** the PDO connection exists. If initialize already creates `$pdo`, use that same connection. Do not create a separate connection for each finder or bird object. Check existing setup before changing it; other asgn07 functionality must remain working.

5.2 📋 Build `public/birds.php` - The directory

Create a plain HTML directory containing **one row for every bird**. Use `Bird::find_all()`, not SQL in the page.

The page must display:

- Common name, scientific name, primary habitat, and seasonal status.
- A **View details** link (or linked common name) pointing to `bird.php?id=<actual id>` for each bird.
- A readable message if `Bird::find_all()` returns no birds.

- A link back to `public/index.php`.

Use an HTML `<table>` with proper headings and `<th scope="col">`; appearance is not graded. Escape all string content coming from the database using your existing `h()` helper **if one exists**, or `htmlspecialchars($value, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8')`. For links, convert the integer ID to an integer before placing it in the URL; do not concatenate untrusted text into an attribute.

Test: You should have eight rows for the original seed (or more if you added birds). Every details link should contain the correct ID from its own record, not a manually numbered row position.

5.3 🔎 Build `public/bird.php` - One bird's details

A directory is useful only if visitors can follow a link for more information. Read the `id` query parameter (for example, `bird.php?id=5`). Validate that it is a **positive integer** before calling your model, then use `Bird::find_by_id()`.

When found, display **all seven** bird properties with appropriate labels (common name, scientific name, family, habitat, season, description, and record ID). Escape database-derived text and include a link back to the directory.

When the ID is missing, `abc`, zero, or negative, return a friendly **400** (bad request) page. When a well-formed positive ID is not present in the database, return a friendly **404** (not found) page. Set `http_response_code()` before sending HTML, and do not show stack traces or raw SQL errors to visitors.

Tests to run:

```
bird.php?id=<ID copied from your directory>  --> Correct bird details
bird.php?id=abc                              --> Friendly invalid-ID message
(400)
bird.php?id=0                                --> Friendly invalid-ID message
(400)
bird.php?id=2147483647                       --> Friendly not-found message
(404)
```

Your own database may have different IDs; **copy the valid ID from your directory** rather than assuming a particular bird is ID 1 or 3.

5.4 🔄 Refactor a small part of asgn07

We do **not** want to rebuild the entire asgn07 application or follow every refactoring from the video series. Make just these targeted improvements:

1. **Inside the model:** Make `find_all()` and `find_by_id()` share the `from_row()` hydration method, as described above.
2. **On `public/index.php`:** Keep the useful asgn07 database summary and existing links. If the index currently counts bird rows with a direct `SELECT COUNT(*)`, replace **only the `birds` count** with `Bird::count_all()`. Keep the existing `bird_images` and `bird_image_links` counts as they are.

3. Add a visible navigation link from `public/index.php` to `birds.php`. Verify the links back to the index and between the directory and detail page.

Add two short sentences to your README: **what you moved/refactored** and **why the new placement is an improvement**. We are *not* introducing a reusable base ActiveRecord class, a generic SQL query engine, or new design frameworks this week.

5.5 🌿 Git branch, commit, merge, and GitHub

Use the **existing** course Git repository; do not accidentally create a second `.git` folder inside asgn08. Run the following commands in the repository root. The example assumes `main` is your primary branch and `origin` is already connected to GitHub.

Before editing:

```
git status
git switch main
git pull --ff-only origin main
git switch -c dev-active-record
```

Copy the working asgn07 project to asgn08 and extract the provided starter files **after the branch is created**, or make sure those new files are first committed on `dev-active-record`. Save meaningful commits as you complete the tasks. For example:

```
git add .
git commit -m "asgn08: add active record sandbox and Bird model"

# After the directory and detail page work:
git add .
git commit -m "asgn08: add directory and detail page"
```

Before staging: Run `git status` and confirm you are **not** about to commit any database usernames/passwords or local-only credentials. `.gitignore` does not remove files that Git already tracks. If a secret was ever pushed to GitHub, removing the file alone is not enough; rotate the credential and seek instructor help.

After the code passes tests, preserve the development branch and merge into main:

```
git push -u origin dev-active-record

git switch main
git merge --no-ff dev-active-record
git push origin main

git status
```

If Git reports a merge conflict, resolve it and test again before pushing. Your GitHub repository must show your final asgn08 code on **main**. Also keep the `dev-active-record` branch on GitHub so the instructor can see that you practiced the workflow.

5.6 📄 Update `.gitignore` and `README.md`

`.gitignore`: Keep the ignore rules you already use. Add rules for any **actual local credentials file** from asgn07, system files, and log files as appropriate; do **not** ignore all of `private/`, your PHP source, or the working sandbox. An *example*, which you must adjust to match your own file names:

```
.DS_Store
*.log
private/credentials.php
```

Do not create or migrate to `.env` in this assignment. We will handle environment files in the next assignment.

`README.md`: Update the README at the top of asgn08. Include the project purpose; use of PHP, PDO, and `wnc_birds`; a short folder map; how to run locally and on the webhost **without exposing credentials**; what `find_all()`, `find_by_id()`, and `from_row()` do; your two-sentence refactoring explanation; and your Git branch/merge workflow. Include navigation URLs. Keep the README short enough to be useful to someone opening the repository for the first time.

5.7 🌐 Upload your site to your webhost

Your PHP application must run on the webhost that you set up earlier in the semester. It is not sufficient to submit only working code on GitHub.

1. Create the destination **`web250/asgn08`** under the webhost directory structure you have been using (for example, `public_html/web250/asgn08`). Upload the working site files, preserving paths and the relative includes needed by `public/`.
2. Create/select a MySQL database through your host's database panel, and import the needed `wnc_birds` tables and seed data **without deleting other work**. Hosting databases often have account-prefixed names, different users, and different MySQL hostnames/ports. When importing the asgn07 SQL into a host-created database, check whether your host requires you to omit the original `CREATE DATABASE` and `USE wnc_birds` statements. Update your host-specific PHP credentials **privately**, using the existing asgn07 credentials approach. Do not place credentials in GitHub or in the README.
3. Protect `private/` from direct web access. Prefer placing private configuration outside the web-accessible document root if your hosting setup supports that, and adjust include paths as necessary. If it must be inside the document root, use the host's documented access controls; on an Apache 2.4 server a `private/.htaccess` containing `Require all denied` can be appropriate, but confirm your hosting server actually honors it. PHP `require_once` can still read those files on the server.
4. Test the **hosted** `public/index.php`, `public/birds.php`, and multiple working `public/bird.php?id=...` links. Also test invalid and missing IDs.

5. Update the **web250 course index file** on your webhost so it links to your **asgn08** site (and does not break the old asgn01-asgn07 links). Update the **asgn08** `public/index.php` file so visitors can reach the new bird directory. Update your local copies as well.
6. Do **not** upload development logs, temporary `phpinfo()` pages, SQL credentials, or private test files into the public portion of the site. The separate `debugging_setup` assignment explains its own safe deployment rules.

AI use for index files: "You can use AI to write the index files because it is grunt work." This permission applies to making/updating the navigation and index-link pages. The Bird model, directory, and detail page are the programming work you must understand and complete yourself. You may ask AI to explain a PHP error or review code you wrote.

A hosted URL may look something like this (use **your own** domain):

```
https://your-domain.example/web250/asgn08/public/birds.php
```

Make sure this URL is reachable **without** an instructor logging into your hosting account.

5.8 📩 Submit in Moodle

In the Moodle assignment's Comments section, paste **both** of these URLs:

```
GitHub (main branch): https://github.com/YOUR-USERNAME/YOUR-
REPOSITORY/tree/main/asgn08
Webhost directory:    https://YOUR-DOMAIN/web250/asgn08/public/birds.php
```

Replace both examples with your actual links. Your repository may have a different root layout; link to the folder that contains your work on `main`. The GitHub link must be accessible to the instructor; the webhost link must load the **live** directory page. No screenshot is required. Your README and Git history must be available through GitHub.

✅ Final student checklist

- ☐ asgn07 still works, and asgn08 is a separate working copy.
- ☐ Guided sandbox runs successfully and is retained as a reference (not graded).
- ☐ `Bird::find_all()` returns an array of `Bird` objects.
- ☐ `Bird::find_by_id()` uses a bound parameter and returns a `Bird` or `null`.
- ☐ Both finders use `from_row()`; `Bird::count_all()` returns an integer.
- ☐ Directory, detail pages, invalid IDs, and not-found IDs all behave correctly.
- ☐ README and `.gitignore` are updated; credentials are not on GitHub.
- ☐ Work was done on `dev-active-record`, merged into `main`, and pushed to GitHub.
- ☐ Webhost site and the web250 / asgn08 index links work.
- ☐ Moodle Comments include **GitHub and webhost URLs**.

💯 Grading rubric (100 points)

Category	Expectations	Points
Active Record class and queries	Correct typed properties, PDO sharing, `find_all()`, safe prepared `find_by_id()`, and `count_all()`.	**30**
Object hydration and modest refactoring	Correct `Bird` objects; `from_row()` reused by both finders; prior index's bird count migrated without breaking asgn07 functionality; sensible separation of SQL/HTML.	**20**
Directory and detail pages	Full directory with working links, seven-field details, escaping, valid ID handling, empty and missing cases, and friendly responses.	**25**
Git feature branch and GitHub	`dev-active-record` work, meaningful commits, merge to `main`, both branches pushed, final code available on GitHub.	**10**
README and `.gitignore`	Clear setup/explanations/refactoring notes and reasonable ignore rules, no committed secrets.	**5**
Webhost deployment and index links	Live site works, web250 and asgn08 index pages are updated, and live details/invalid-ID tests pass.	**8**
Moodle submission links	Both correct GitHub and webhost URLs are in Moodle Comments.	**2**
TOTAL		**100**

Separate 100-point activity: Complete the companion **`debugging_setup_instructions.md`** using its separate starter ZIP and Moodle activity; it is not a prerequisite for deploying the finished asgn08 site, but it will help you debug your PHP applications.

Scope reminder: We are learning how to build a reliable, **read-only Active Record directory**. Do not add CRUD forms, delete existing tables, introduce a framework, or create `.env` files for asgn08. A plain, correct PHP application is the goal.